



Universidad
Nacional
de Loja

UNIVERSIDAD NACIONAL DE LOJA

**FACULTAD DE ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES NO
RENOVABLES**

CARRERA DE COMPUTACIÓN

MATEMATICAS DISCRETAS

TITULO DEL TRABAJO:

“TRABAJO AUTONOMO DE TEORIA DE GRAFOS Y ARBOLES”

ESTUDIANTE:

Andy Ordoñez

Miguel Rojas

David Luna

Karen Lalangui

DOCENTE:

Ing. Mario Cueva

FECHA:

17/07/2026

REPORTE TÉCNICO DE ACTIVIDADES PRÁCTICO-EXPERIMENTALES N.º 3**1. DATOS DE IDENTIFICACIÓN DEL ESTUDIANTE Y LA PRÁCTICA**

Estudiantes	Miguel Rojas, David Luna, Andy Ordoñez y Karen Lalangui.
Asignatura	Matemáticas Discretas
Ciclo	1er ciclo
Unidad	Unidad 3: Grafos y Árboles
Resultado de aprendizaje de la unidad	Entiende los principios del análisis combinatorio que resuelven problemas reales de su cotidianidad, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	3
Título de la Práctica	Teoría de Grafos, Algoritmos de Búsqueda, Árboles y Aplicaciones Computacionales
Fecha	29 de junio al 24 de julio de 2026
Horario	Lunes, Martes, jueves 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Laboratorio de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	12 horas (3 horas por sesión APE)

2. OBJETIVO(S) DE LA PRÁCTICA

- ✓ Comprender los fundamentos de la teoría de grafos: definición, terminología (vértices, aristas, grado, adyacencia, incidencia), tipos de grafos (dirigidos, no dirigidos, ponderados, completos, bipartitos) y representaciones (matriz de adyacencia, lista de adyacencia).

- ✓ Aplicar algoritmos fundamentales de búsqueda en grafos: BFS (Búsqueda a lo Ancho), DFS (Búsqueda en Profundidad) y Dijkstra (camino más corto), comprendiendo su funcionamiento paso a paso.
- ✓ Entender la teoría de árboles: definición, propiedades, tipos de árboles (binarios, BST, AVL), recorridos (inorden, preorden, posorden) y aplicaciones en ciencias de la computación.
- ✓ Utilizar software de modelado de grafos (Graph Online, Gephi) para representar, visualizar y analizar redes y estructuras de datos.
- ✓ Integrar los conocimientos de las tres unidades mediante un proyecto que aplique la lógica matemática, el álgebra de Boole y la teoría de grafos a un problema computacional real.

3. MATERIALES Y REACTIVOS

- ✓ Pizarra y marcadores de colores.
- ✓ Proyector multimedia.
- ✓ Hojas de trabajo con ejercicios de teoría de grafos y árboles.
- ✓ Guía paso a paso de los algoritmos BFS, DFS y Dijkstra.
- ✓ Plantillas para construcción de grafos y árboles.
- ✓ Investigación sobre aplicaciones de grafos en redes y optimización.

4. EQUIPOS Y HERRAMIENTAS

- ✓ PC o laptop (mínimo 1 por estudiante).
- ✓ Plataforma EVA de la UNL.
- ✓ Laboratorio de computación.
- ✓ Software Graph Online (graphonline.ru/es) o Gephi para modelado de grafos; herramientas colaborativas: Google Drive, Wiki.
- ✓ Software de presentación (PowerPoint, Canva).
- ✓ Máximo de personas por grupo: 4-5 estudiantes.

Procedimiento / Metodología***Fase 1: Fundamentos de la Teoría de Grafos (Semana 13: 29 de junio - 3 de julio de 2026)***

- ✓ Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (por ejemplo, modelar la red de amigos de un grupo social, la red de transporte de una ciudad o la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.
- ✓ Modelado de una red sencilla mediante software Graph Online o Gephi: los equipos crean un grafo representando una red de transporte en la ciudad de Loja, visualizándolo con el software y analizando sus propiedades (conectividad, caminos mínimos, nodos críticos).

Fase 2: Algoritmos de Búsqueda y Caminos Mínimos (Semana 14: 6-10 de julio de 2026)

- ✓ Taller colaborativo de aplicación de algoritmos: los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra), documentando cada paso: cola/pila de nodos visitados, orden de descubrimiento, distancias acumuladas y caminos resultantes.
- ✓ Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos, comparando los resultados manuales con los obtenidos por el software.

Fase 3: Árboles y Estructuras de Datos Arbóreas (Semana 15: 13-17 de julio de 2026)

- ✓ Los estudiantes construyen árboles binarios de búsqueda (BST) paso a paso, insertando valores numéricos y realizando los tres tipos de recorridos, documentando el orden de visita de nodos en cada caso.
- ✓ Los estudiantes investigan de forma autónoma sobre la relación entre la teoría de grafos/árboles y las estructuras de datos utilizadas en programación (heaps, tries, árboles B), bases de datos (índices) e inteligencia artificial (árboles de decisión). Elaboran un miniinforme con exposición apoyada en recursos TIC.

Fase 4: Proyecto Integrador - Modelado y Optimización de Redes (Semana 16: 20-24 de julio de 2026)

- ✓ Los equipos elaboran un informe técnico del proyecto que incluye: descripción del problema, modelado matemático (lógica, booleana, grafos), implementación en software, resultados obtenidos, análisis de optimización y conclusiones (artículo).

Fase 5: Exposición, Defensa y Evaluación Final (Semana 17: 27-31 de julio de 2026)

- ✓ Exposición y defensa oral del proyecto integrador ante la clase: cada equipo presenta su trabajo (10-15 minutos), explicando el problema abordado, la metodología aplicada (lógica + booleana + grafos), los resultados obtenidos y las conclusiones.
- ✓ **Reflexión final:** los estudiantes elaboran una autoevaluación individual sobre su proceso de aprendizaje durante las tres unidades, identificando fortalezas, debilidades y metas de mejora para el siguiente ciclo académico (individual).

Resultados

FASE 1: FUNDAMENTOS DE LA TEORÍA DE GRAFOS

Ejemplo 1: La Casa de Edward A. Mouse

Contextualización

Edward A. Mouse acaba de terminar su nueva casa. El plano se muestra a continuación:

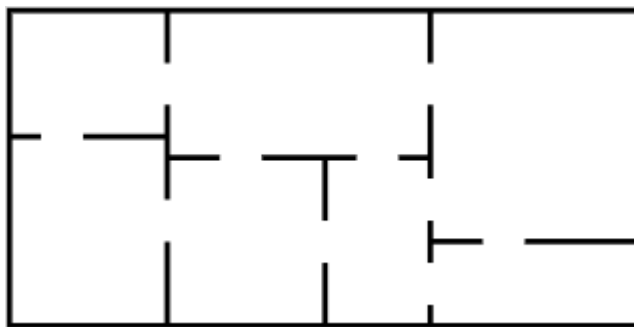


Figura 1. Plano de la casa de Edward A. Mouse.

Para el desarrollo del ejercicio se consideran las habitaciones como vértices y las puertas como aristas. Como primer paso se identifica el número de habitaciones: el problema presenta 7

habitaciones. Por otro lado, las puertas representan el número de conexiones entre dos habitaciones.

Representación Gráfica

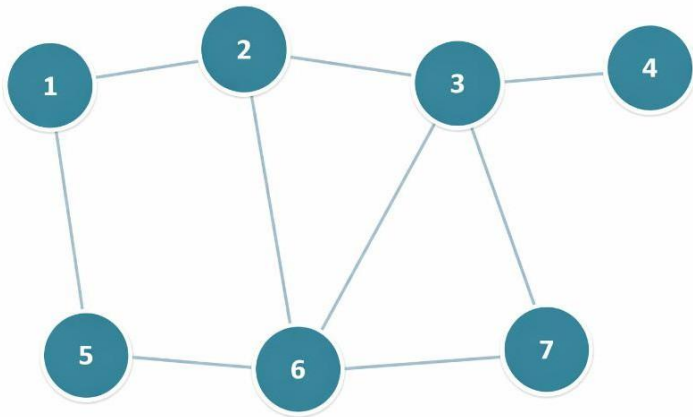


Figura 2. Representación gráfica del grafo de la casa.

Número de Grados

Tabla 1. Representación de los vértices y número de grados.

Vértice	Grado
H1	2
H2	3
H3	4
H4	1
H5	2
H6	4
H7	2

Desarrollo

a. ¿Es posible atravesar cada puerta exactamente una vez? Sí. Existe un camino euleriano, ya que todos los vértices tienen grado par, excepto dos, que pueden tener grado impar, por lo que se pueden recorrer todas las puertas exactamente una vez. El recorrido debe empezar en una habitación de grado impar y terminar en la otra.

b. ¿Es posible visitar cada habitación exactamente una vez sin usar todas las puertas? Aquí no importa usar todas las puertas; solo interesa visitar cada habitación una sola vez. Por lo tanto, sí existe un camino hamiltoniano, por lo que se puede visitar cada habitación exactamente una vez, sin necesidad de usar todas las puertas.

c. ¿Es posible que cada habitación tenga un número impar de puertas tras la remodelación? Sí. Es posible agregar puertas entre habitaciones para que todas tengan un número impar de puertas, ya que la paridad de dos habitaciones cambia con cada puerta añadida y el número de habitaciones es par.

Propiedades Estructurales

- Es un grafo no dirigido, porque las puertas permiten el paso entre dos habitaciones en ambos sentidos; no existe una dirección obligatoria para recorrerlas.
- Es un grafo conexo, ya que es posible desplazarse desde cualquier habitación hasta otra siguiendo las puertas disponibles.
- Presenta dos vértices de grado impar (H2 y H4) y cinco vértices de grado par. Esta característica permite la existencia de un camino euleriano, que comienza en uno de los vértices de grado impar y termina en el otro.

Ejemplo 2: La Constelación de Orión

Contextualización

Como ejemplo de aplicación de la teoría de grafos, se seleccionó la constelación de Orión, una de las constelaciones más conocidas del cielo nocturno. En este modelo, cada estrella se representa mediante un vértice, mientras que las líneas imaginarias que tradicionalmente unen las estrellas para formar la figura de la constelación se representan mediante aristas. Es importante

señalar que estas aristas no representan conexiones físicas entre las estrellas, sino únicamente una representación gráfica utilizada en astronomía para identificar la forma de la constelación.

Para este modelo se consideran las estrellas Betelgeuse, Bellatrix, Alnitak, Alnilam, Mintaka, Saiph y Rigel. Las conexiones entre ellas siguen el patrón utilizado para representar la figura principal de Orión.

Representación Gráfica

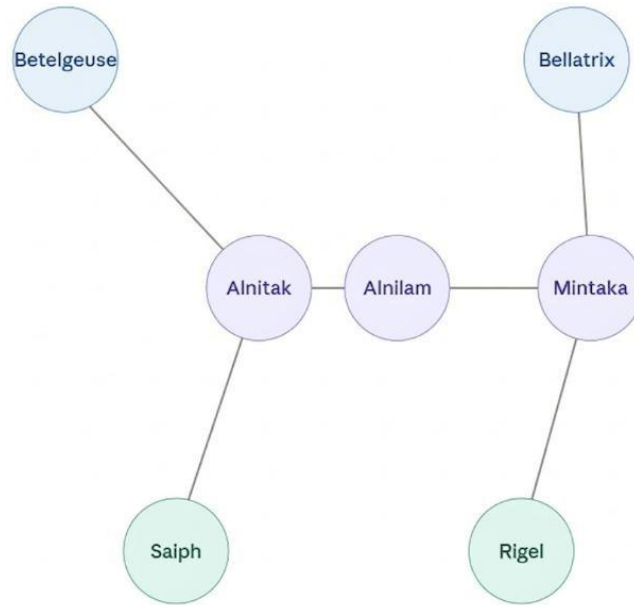


Figura 3. Representación gráfica de la constelación de Orión como grafo.

Aristas (Conexiones)

- Betelgeuse - Alnitak
- Bellatrix - Alnitak
- Alnitak - Alnilam
- Alnilam - Mintaka
- Alnitak - Saiph
- Mintaka - Rigel

Tabla 2. Representación de los vértices y número de grados.

Vértice	Grado
Betelgeuse	1
Bellatrix	1
Alnitak	3
Alnilam	2
Mintaka	3
Saiph	1
Rigel	1

Propiedades Estructurales

- Es un grafo no dirigido, porque las líneas no tienen dirección.
- Es conexo, ya que todas las estrellas están conectadas por algún camino.
- No es un grafo completo, porque no todas las estrellas están conectadas entre sí.

Ejemplo 3: Modelado de la Red de Transporte de la Ciudad de Loja

Contextualización

Para el desarrollo de esta actividad, se seleccionó y cargó un modelo de red preexistente (estructura poligonal de 6 vértices) desde la base de datos de ejemplos de la plataforma digital Graph Online. Este modelo se utilizó como base topológica para simular el comportamiento del flujo vehicular, las distancias y la conectividad en la ciudad de Loja.

Estructura y Mapeo del Grafo Seleccionado

El grafo cargado cuenta con 6 vértices (identificados del 0 al 5), los cuales han sido mapeados conceptualmente para representar puntos estratégicos de la infraestructura vial de la urbe lojana:

- Nodo 0: Terminal Terrestre (TT).
- Nodo 1: Complejo Ferial Jipiro (CFJ).
- Nodo 2: Puerta de la Ciudad (PC).

- Nodo 3: Universidad Técnica Particular de Loja (UTPL).
- Nodo 4: Parque Central (PCE).
- Nodo 5: Universidad Nacional de Loja / Campus Argelia (UNL).

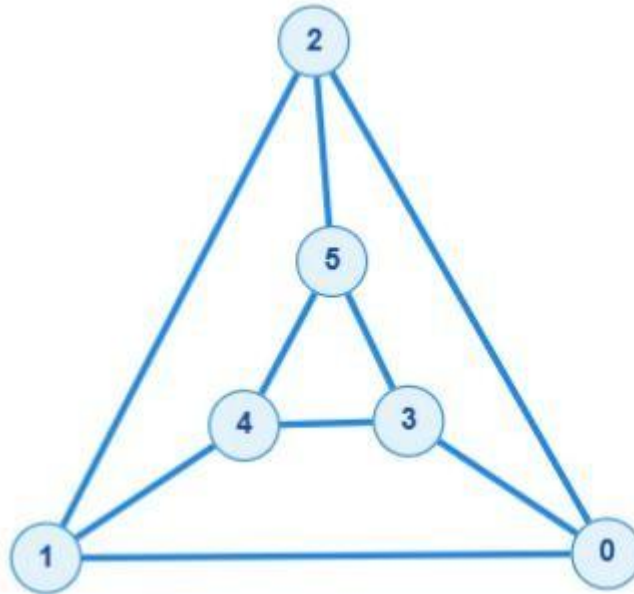


Figura 4. Grafo de la red de transporte de la ciudad de Loja.

Análisis de Propiedades Estructurales de la Red

Al procesar la plantilla seleccionada mediante las herramientas algorítmicas del software, se obtuvieron los siguientes resultados analíticos:

Conectividad

Al ejecutar la verificación en el programa, se determina que el grafo es estrictamente conexo. No existen componentes o subredes aisladas. Esto demuestra que la red vial modelada permite el flujo ininterrumpido de transporte, garantizando que un vehículo pueda partir desde cualquier origen (como el Terminal Terrestre en el norte) y alcanzar cualquier destino (como la UNL en el sur) empleando las avenidas y conexiones existentes.

Caminos Mínimos (Optimización de Rutas)

Utilizando la herramienta para encontrar el camino más corto (algoritmo de Dijkstra) entre el Terminal Terrestre (Nodo 0) y el Parque Central (Nodo 4), el sistema evalúa y compara los trayectos periféricos disponibles:

- *Ruta occidental:* pasando directamente por el Nodo 2 (Nodo 0 → Nodo 2 → Nodo 4).
- *Ruta oriental:* fluyendo por los Nodos 1 y 3 (Nodo 0 → Nodo 1 → Nodo 3 → Nodo 4).

El software resalta visualmente la trayectoria a través del eje occidental (Nodo 2) como el camino mínimo óptimo, ya que reduce la cantidad de tramos viales intermedios y optimiza los recursos y tiempos necesarios para el traslado de pasajeros.

Nodos Críticos (Puntos de Articulación)

El análisis topológico del entorno gráfico identifica los siguientes puntos de vulnerabilidad operativa en el sistema de transporte:

- ✓ *Hub de distribución (Nodo 5 - UNL):* en este diseño preestablecido, el Nodo 5 se ubica en el centro geométrico conectando directamente con los Nodos 2 y 3. Actúa como un interceptor de tráfico masivo. Si ocurre un bloqueo en este nodo, se pierde una vía rápida de distribución urbana secundaria.
- ✓ *Puntos de corte:* al ser una red perimetral cerrada con un núcleo central, los nodos periféricos con menor conectividad cruzada dependen enteramente de sus enlaces contiguos. Si se inhabilitan las aristas principales que alimentan al Nodo 4 (Parque Central), este sector colapsa, alterando la distribución fluida hacia los extremos de la ciudad.

Resolución: Ejecución Manual

FASE 2: ALGORITMOS DE BÚSQUEDA Y CAMINOS MÍNIMOS

BFS (Búsqueda a lo Ancho) - Grafo No Dirigido

Se utilizó un grafo simple con 4 nodos: A-B, A-C, B-D, C-D. El algoritmo BFS utiliza una estructura de datos FIFO (First In, First Out), es decir, una cola. El orden de descubrimiento obtenido fue: A, B, C, D.

Tabla 3. Ejecución manual del algoritmo BFS.

Iteración	Nodo actual	Vecinos no visitados	Estado de la cola (nodos visitados)
Inicio	Ninguno	Ninguno	[A]
1	A	B, C	[B, C]
2	B	D	[C, D]
3	C	Ninguno (D ya está en cola)	[D]
4	D	Ninguno	[] (vacía)

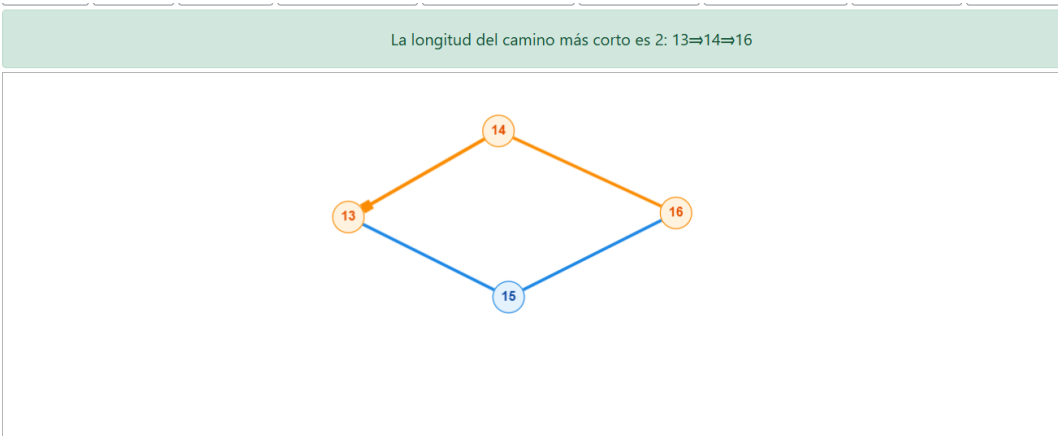


Figura 5. Grafo utilizado para BFS, validado en software.

DFS (Búsqueda en Profundidad) - Grafo No Dirigido

Usando el mismo grafo anterior (A-B, A-C, B-D, C-D), el algoritmo DFS utiliza una estructura LIFO (Last In, First Out), es decir, una pila. El orden de descubrimiento obtenido fue: A, B, D, C.

Tabla 4. Ejecución manual del algoritmo DFS.

Iteración	Nodo actual	Vecino a explorar	Estado de la pila (ruta de exploración)
Inicio	Ninguno	Ninguno	[A]
1	A	B	[A, B]
2	B	D	[A, B, D]
3	D	C	[A, B, D, C]
4	C	Ninguno (fin de rama)	[A, B, D] (retroceso)



Figura 6. Grafo utilizado para DFS, validado en software.

Dijkstra (Camino Más Corto) - Grafo Ponderado

Se analizó la latencia entre servidores, considerando 3 nodos con los siguientes pesos (latencia): $A-B = 4$, $A-C = 1$, $C-B = 2$. Se buscó el camino más corto desde A hasta B. El camino óptimo resultante es $A \rightarrow C \rightarrow B$, con una distancia acumulada total de 3.

Tabla 5. Ejecución manual del algoritmo de Dijkstra.

Nodo	Distancia acumulada	Nodo previo (ruta)	¿Visitado definitivamente?
A	0	Ninguno	Sí
C	1	A	Sí
B	3 (a través de C)	C	Sí

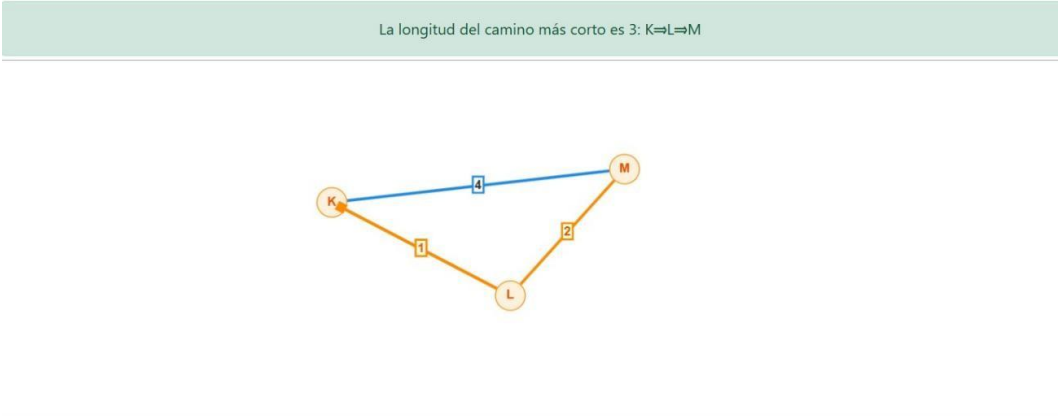


Figura 7. Grafo ponderado utilizado para Dijkstra, validado en software.

Comparación: Ejecución Manual vs. Software de Grafos

Esta sección presenta la comparación entre los resultados obtenidos mediante la ejecución manual de algoritmos de grafos (BFS, DFS y Dijkstra) y la validación utilizando el software de modelado Graph Online, cumpliendo con el requerimiento de la actividad colaborativa.

Algoritmo de Dijkstra (Camino Más Corto en Grafo Ponderado)

Se evaluó un grafo ponderado para encontrar la latencia mínima entre servidores.

Criterio	Ejecución manual	Resultados del software (Graph Online)
Nodos / vértices	A, B, C	K, L, M
Pesos (latencia)	$A-B = 4$; $A-C = 1$; $C-B = 2$	$K-M = 4$; $K-L = 1$; $L-M = 2$
Ruta óptima calculada	$A \rightarrow C \rightarrow B$	$K \rightarrow L \rightarrow M$
Distancia total	3	3
Conclusión	Los resultados manuales coinciden exactamente con los generados por el software. La ruta calculada iterativamente minimiza el peso total ($1 + 2 = 3$) frente a la arista directa (peso 4), validando correctamente la matriz de distancias.	

Búsqueda a lo Ancho (BFS) - Grafo No Dirigido

Se analizó la exploración por niveles utilizando una estructura de datos FIFO (cola).

Criterio	Ejecución manual	Resultados del software
Nodos	A, B, C, D	6, 7, 8, 9
Propósito / estado	Exploración en anchura. Orden: A, B, C, D.	Búsqueda del camino más corto sin pesos.
Camino resultante	A → C (directo en nivel 1)	6 → 9 (longitud del camino: 1)
Conclusión	El software confirma que, en un grafo no ponderado, los nodos adyacentes descubiertos en la primera iteración de la cola (como A-C o 6-9) representan automáticamente el camino más corto con una distancia de 1 salto.	

Búsqueda en Profundidad (DFS) - Grafo No Dirigido

Se analizó la exploración profunda por ramas utilizando una estructura LIFO (pila).

Criterio	Ejecución manual	Resultados del software
Estructura utilizada	Pila (retroceso/backtracking)	Navegación perimetral del grafo
Ruta de exploración	A, B, D, C	15 → 16 → 14 → 13
Conclusión	El rastro manual en pila [A, B, D, C] representa de forma correcta la inmersión hasta el final de una rama antes de realizar el retroceso, lo cual es equivalente gráficamente a la traza de rutas continuas simulada por el software.	

Reporte de Comparación: Ejecución Manual vs. Software - Algoritmo de Dijkstra

Esta sección presenta la resolución y comparación de un nuevo grafo ponderado ingresado, aplicando el algoritmo de Dijkstra para encontrar el camino más corto entre el nodo de origen (18) y el nodo de destino (24).

La longitud del camino más corto es 22: 18⇒21⇒23⇒24

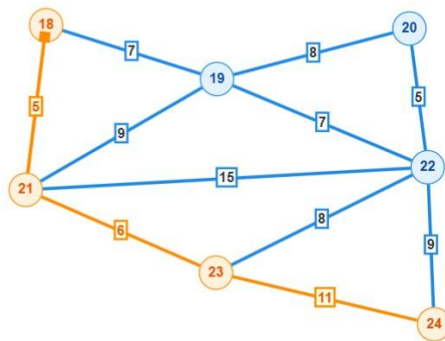


Figura 8. Grafo ponderado ingresado en el software (nodo 18 a nodo 24).

Ejecución Manual Paso a Paso (Algoritmo de Dijkstra)

A continuación, se detalla el estado de las distancias acumuladas y nodos previos en cada iteración del algoritmo, partiendo desde el nodo 18.

Tabla 6. Ejecución manual paso a paso del algoritmo de Dijkstra (nodo 18 a nodo 24).

Iteración	Nodo evaluado	Actualización de vecinos (distancia acumulada, nodo previo)	Nodos visitados
Inicio	18	19 (7, prev: 18); 21 (5, prev: 18)	{18}
1	21	22 (5+15=20, prev: 21); 23 (5+6=11, prev: 21)	{18, 21}
2	19	20 (7+8=15, prev: 19); 22 (7+7=14, prev: 19) → se actualiza porque 14 < 20	{18, 21, 19}
3	23	24 (11+11=22, prev: 23); 22 (11+8=19) → no se actualiza, 14 es menor	{18, 21, 19, 23}

Iteración	Nodo evaluado	Actualización de vecinos (distancia acumulada, nodo previo)	Nodos visitados
4	22	24 (14+9=23) → no se actualiza, 22 es menor; 20 (14+5=19) → no se actualiza, 15 es menor	{18, 21, 19, 23, 22}
5	20	Sin actualizaciones viables a nodos no visitados.	{18, 21, 19, 23, 22, 20}
6	24	Destino alcanzado.	{18, 21, 19, 23, 22, 20, 24}

Camino resultante manual reconstruido: $24 \leftarrow 23 \leftarrow 21 \leftarrow 18$ (distancia total: $5 + 6 + 11 = 22$).

Tabla Comparativa: Manual vs. Software

Criterio	Ejecución manual	Resultados del software (Graph Online)
Nodo origen → destino	18 → 24	18 → 24
Ruta óptima calculada	18 → 21 → 23 → 24	18 → 21 → 23 → 24
Distancia total (latencia/peso)	22 (5 + 6 + 11)	22
Conclusión de validación	Los resultados manuales coinciden exactamente con los generados por el software. La traza algorítmica demuestra que, a pesar de existir múltiples rutas hacia el nodo 24 (por ejemplo, 18-19-22-24 con peso 23, o 18-21-22-24 con peso 29), la evaluación por pesos acumulados descarta correctamente los subcaminos ineficientes.	

FASE 3: ÁRBOLES Y ESTRUCTURAS DE DATOS ARBÓREAS

Basado en el capítulo 7 (que inicia en la página 241) del libro Matemáticas Discretas: Aplicaciones y Ejercicios, de José Francisco Villalpando Becerra, el análisis de los recorridos en profundidad de un árbol binario se estructura mediante una nomenclatura específica.

El autor define la metodología basándose en el orden exacto en el que el algoritmo visita tres elementos: el nodo raíz (N), el subárbol izquierdo (I) y el subárbol derecho (D).

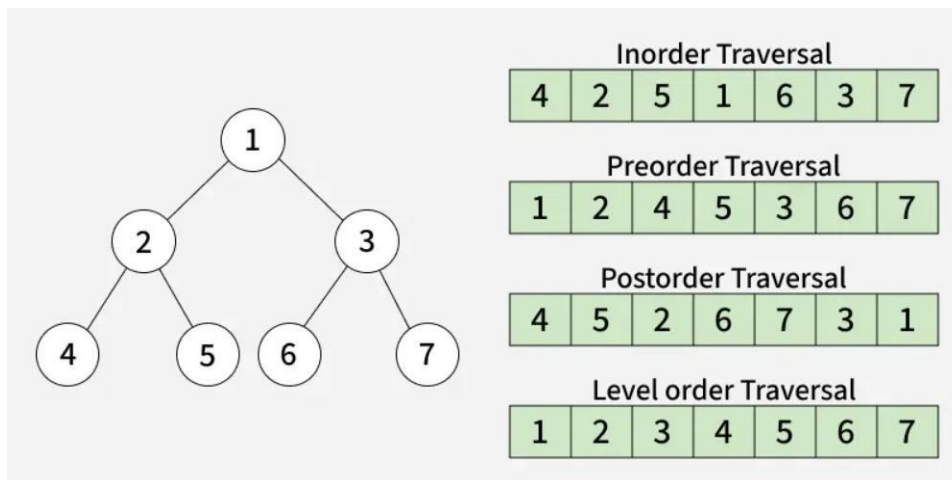


Figura 9. Nomenclatura de los recorridos en profundidad (N, I, D).

Resolución Analítica Paso a Paso

Para ilustrar la metodología, se utiliza la estructura del árbol de ejemplo del texto de referencia (Figura 19 del libro). Consiste en un árbol con la raíz principal r, un subárbol izquierdo encabezado por a, y un subárbol derecho encabezado por b:

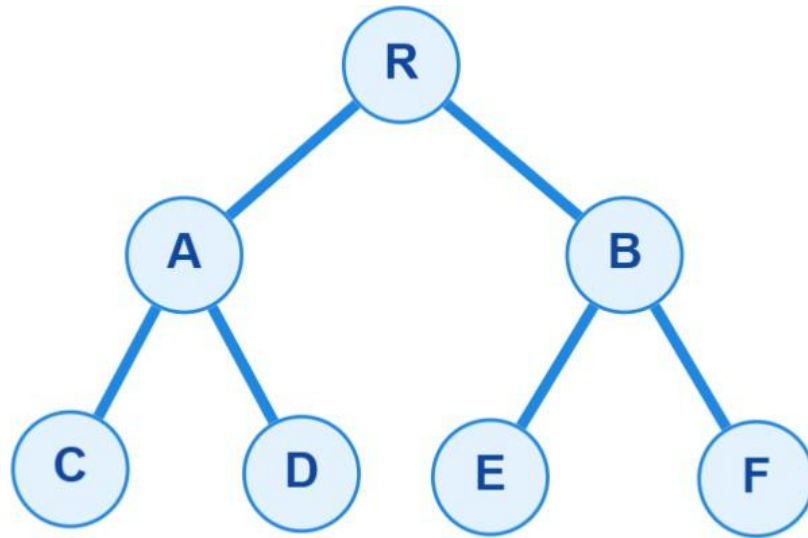


Figura 10. Árbol de ejemplo con raíz r , subárbol izquierdo a y subárbol derecho b .

Recorrido en Preorden (NID)

Regla metodológica: nodo - izquierdo - derecho (NID). Se visita primero la raíz, luego se recorre todo el subárbol izquierdo aplicando la misma regla, y finalmente el derecho.

- Paso 1 (N): se visita la raíz principal: r .
- Paso 2 (I): se pasa al subárbol izquierdo (raíz a). Se aplica NID localmente: se visita el nodo a , luego su hijo izquierdo c , y después su hijo derecho d .
- Paso 3 (D): se pasa al subárbol derecho (raíz b). Se aplica NID localmente: se visita el nodo b , luego su hijo izquierdo e , y después su hijo derecho f .

Secuencia final: $r - a - c - d - b - e - f$

Recorrido en Enorden / Inorden (IND)

Regla metodológica: izquierdo - nodo - derecho (IND). Se recorre el subárbol izquierdo hasta su extremo, se visita la raíz, y luego se pasa al derecho.

- Paso 1 (I): en el subárbol izquierdo de r , se baja hasta el nodo más a la izquierda: c .
- Paso 2 (N): se sube a visitar su nodo padre/raíz local: a .
- Paso 3 (D): se visita el hijo derecho de a : d .
- Paso 4 (N): con el subárbol izquierdo completo, se visita la raíz principal del árbol: r .

- Paso 5 (I, N, D): se aplica la regla al subárbol derecho: primero el izquierdo (e), luego su nodo padre (b), y al final el derecho (f).

Secuencia final: c - a - d - r - e - b - f

Recorrido en Postorden (IDN)

Regla metodológica: izquierdo - derecho - nodo (IDN). Se procesan primero las ramas inferiores (de izquierda a derecha) y la evaluación de la raíz se deja estrictamente para el final.

- Paso 1 (I): en el subárbol izquierdo, se procesan primero las hojas y al final su raíz local: izquierdo (c), derecho (d), nodo (a).
- Paso 2 (D): en el subárbol derecho, se aplica la misma lógica: izquierdo (e), derecho (f), nodo (b).
- Paso 3 (N): finalmente, se visita la raíz principal de todo el árbol: r.

Secuencia final: c - d - a - e - f - b - r

Resolución de Ejercicios

Ejercicio 1: Árbol de Expresión Aritmética

Los árboles binarios se usan comúnmente para evaluar expresiones matemáticas. Este árbol representa la expresión $(A + B) * (C - (D / E))$.

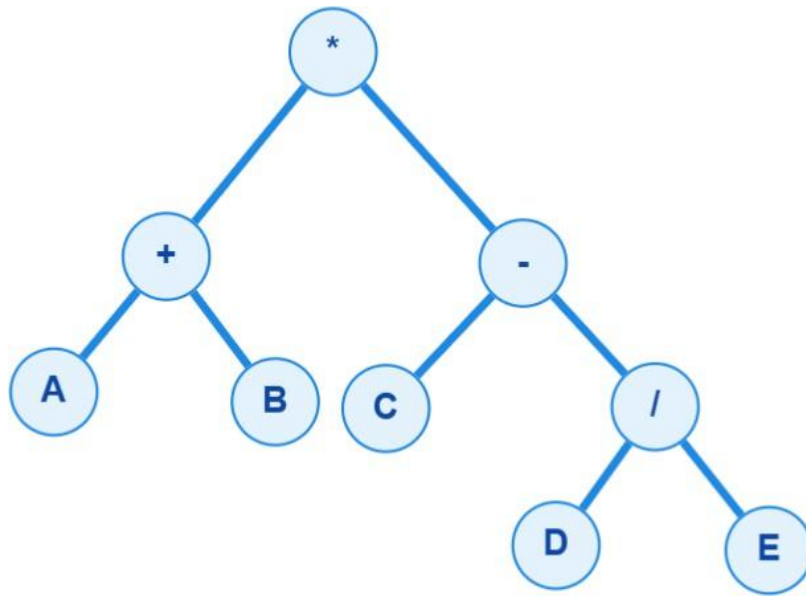


Figura 11. Árbol de expresión aritmética.

Preorden (NID): se empieza en la raíz *, luego todo el lado izquierdo (+ A B), y luego el derecho (- C / D E). Resultado: * + A B - C / D E.

Inorden (IND): extremo izquierdo (A), su padre (+), el derecho (B); luego la raíz principal (*); luego el lado derecho procesado igual. Resultado: A + B * C - D / E. Nota: el inorden en árboles de expresión genera la notación infija estándar, aunque sin paréntesis.

Postorden (IDN): se resuelven las hojas de izquierda a derecha; primero A y B con su nodo +, luego el subárbol derecho (C, D, E, /, -), y al final la raíz *. Resultado: A B + C D E / - *.

Ejercicio 2: Árbol Desbalanceado a la Izquierda

Este árbol tiene una tendencia marcada hacia las ramas izquierdas, lo que cambia la forma en que se percibe el recorrido.

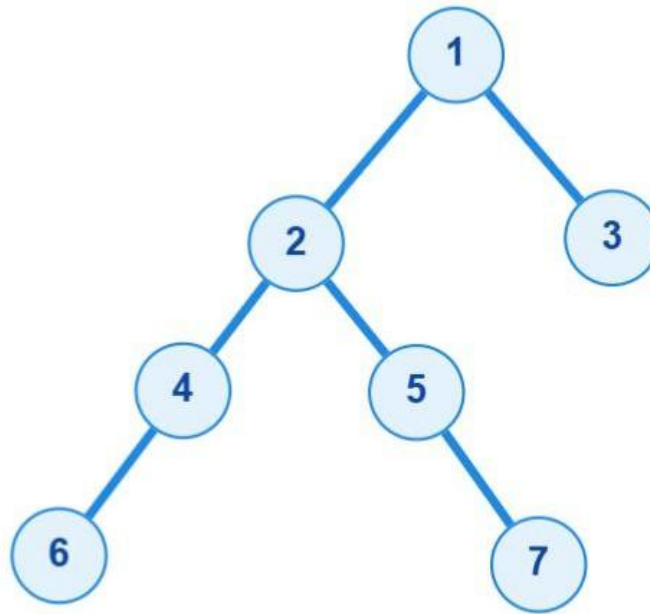


Figura 12. Árbol desbalanceado a la izquierda.

Preorden (NID): 1 (nodo), subárbol izquierdo de 1 (2, 4, 6, 5, 7), y finalmente el hijo derecho 3. Resultado: 1-2-4-6-5-7-3.

Inorden (IND): se baja hasta el 6, se sube a 4, no hay derecho, se sube a 2, se baja al izquierdo de 5 (vacío), nodo 5, derecho 7, raíz 1, derecho 3. Resultado: 6-4-2-5-7-1-3.

Postorden (IDN): se empieza por la hoja 6, se sube a su padre 4; luego se va a la hoja 7, su padre 5, luego el padre 2; luego hoja 3; finalmente, raíz 1. Resultado: 6-4-7-5-2-3-1.

Ejercicio 3: Árbol Binario de Búsqueda (BST) Alfabetizado

En un árbol de búsqueda binaria, el hijo izquierdo es menor que el padre y el derecho es mayor. Se utilizan letras.

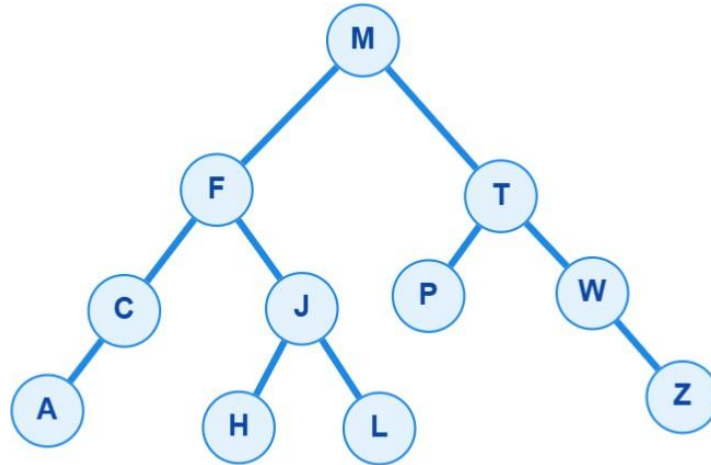


Figura 13. Árbol binario de búsqueda alfabetizado.

Preorden (NID): M (raíz), subárbol izquierdo (F, C, A, J, H, L), subárbol derecho (T, P, W, Z). Resultado: M-F-C-A-J-H-L-T-P-W-Z.

Inorden (IND): como es un árbol de búsqueda, el recorrido inorden siempre devuelve los elementos ordenados (de la A a la Z). Resultado: A-C-F-H-J-L-M-P-T-W-Z.

Postorden (IDN): primero las hojas de la rama izquierda (A, luego C), luego las hojas del subárbol de J (H, L, J), el nodo F, y la misma lógica en el lado derecho. Resultado: A-C-H-L-J-F-P-Z-W-T-M.

Ejercicio 4: Árbol “Zig-Zag”

Este árbol alterna entre hijos únicos izquierdos y derechos, generando una estructura larga pero con pocos nodos.

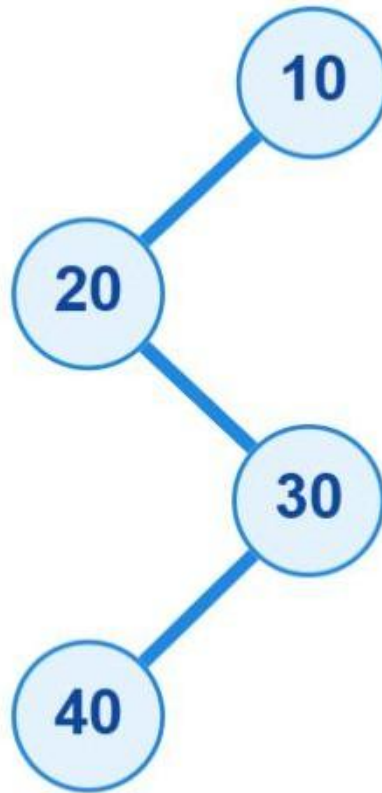


Figura 14. Árbol zig-zag.

Preorden (NID): se visitan los nodos conforme se baja, ya que no hay ramas paralelas.
Resultado: 10-20-30-40.

Inorden (IND): se baja por el 10, luego 20 (tiene izquierdo vacío, así que se visita nodo 20), luego se baja a su derecho 30 (tiene izquierdo 40, así que se visita 40), y se sube a 30.
Resultado: 40-30-20-10.

Postorden (IDN): se baja hasta el último hijo disponible (40), se visita, y se retrocede nodo por nodo hacia la raíz. Resultado: 40-30-20-10.

Ejercicio 5: Árbol Estrictamente Binario (Lleno hasta el Nivel 2)

Es un árbol donde cada nodo tiene exactamente 0 o 2 hijos.

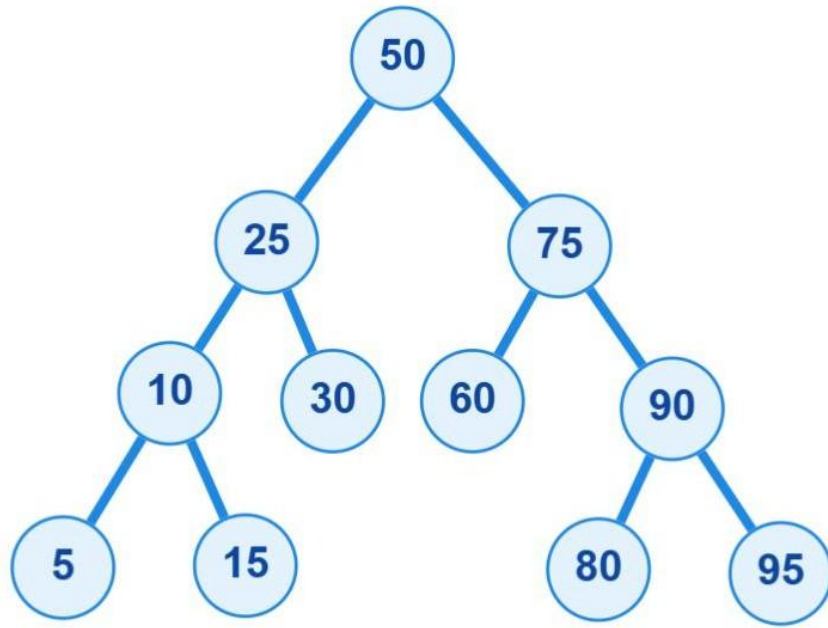


Figura 15. Árbol estrictamente binario.

Preorden (NID): raíz 50, luego se procesa toda la mitad izquierda completa, luego la derecha. Resultado: 50-25-10-5-15-30-75-60-90-80-95.

Inorden (IND): extremo izquierdo (5), padre (10), derecho (15); se sube al padre superior (25), derecho (30); raíz principal 50; mitad derecha. Resultado: 5-10-15-25-30-50-60-75-80-90-95.

Postorden (IDN): se agrupa por hojas inferiores: 5 y 15 con su padre 10, luego el 30, todos bajo el 25; misma lógica en la derecha; raíz al final. Resultado: 5-15-10-30-25-60-80-95-90-75-50.

Miniinforme: Aplicaciones de la Teoría de Árboles en la Computación Moderna

Introducción

En matemáticas discretas, un árbol es un grafo conexo, no dirigido y sin ciclos, compuesto por un conjunto de vértices (nodos) y aristas. Aunque su definición matemática es abstracta, en la informática los árboles proporcionan una jerarquía y una estructura organizativa que permite buscar, insertar y eliminar datos con una eficiencia algorítmica excepcional, superando las limitaciones de las estructuras lineales como los arreglos o las listas.

Aplicación: hacen posible el autocompletado en los teclados de los celulares y los motores de búsqueda, encontrando palabras completas a partir de un prefijo en fracciones de milisegundo.

Árboles B: son árboles de búsqueda auto-balanceados que pueden tener múltiples hijos por nodo, manteniendo los datos ordenados.

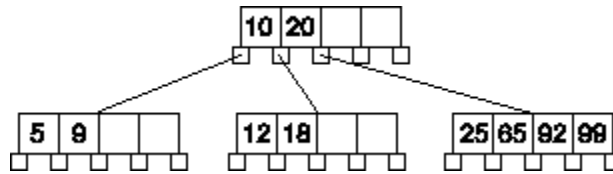


Figura 18. Estructura de un árbol B.

Aplicación: permiten realizar operaciones de búsqueda e inserción en tiempo logarítmico $O(\log n)$, siendo fundamentales cuando se manejan volúmenes masivos de información.

Bases de Datos

Cuando se realiza una consulta en una base de datos con millones de registros, el sistema no revisa fila por fila (escaneo secuencial); en su lugar, utiliza índices construidos casi exclusivamente sobre árboles B o árboles B+.

El mecanismo: la base de datos guarda la información en los nodos hoja del árbol. Al buscar un dato, el motor navega rápidamente por las ramas (nodos internos), descartando mitades enteras de información en cada paso.

El impacto: esto minimiza drásticamente las operaciones de lectura en el disco duro, pasando de millones de lecturas a solo un puñado de saltos a través de las aristas del árbol.

Inteligencia Artificial

En el campo del machine learning, los grafos acíclicos dirigidos toman la forma de árboles de decisión, uno de los algoritmos predictivos más puros y explicables.

Funcionamiento: cada nodo interno representa una pregunta o prueba matemática sobre una característica de los datos (por ejemplo, “¿el valor es mayor a 5?”). Cada rama representa la respuesta a esa prueba, y los nodos terminales (hojas) representan la predicción o clasificación final.

Grafos de estado: adicionalmente, algoritmos de IA clásica (como la navegación de NPC en videojuegos o el análisis de jugadas en ajedrez) utilizan grafos de búsqueda espacial donde cada nodo es un estado posible del mundo.

```

1 import matplotlib.pyplot as plt
2 from sklearn.tree import DecisionTreeClassifier, plot_tree
3
4 # 1. PREPARACION DE LOS DATOS (El contexto)
5 # Formato: [Horas de estudio a la semana, Porcentaje de asistencia]
6 X = [
7     [2, 50], [3, 80], [1, 90], # Estudiantes con pocas horas de estudio
8     [8, 95], [7, 70], [6, 100], # Estudiantes con muchas horas y buena asistencia
9     [4, 60], [5, 50]           # Estudiantes con horas medias pero mala asistencia
10 ]
11
12 # Etiquetas (Las "hojas" del grafo): 0 = Reprueba, 1 = Aprueba
13 y = [0, 0, 0, 1, 1, 1, 0, 0]
14
15 # 2. CREACION DEL MODELO MATEMATICO (El Grafo)
16 # max_depth=3 limita la profundidad del arbol para que sea facil de leer en la exposicion
17 clasificador = DecisionTreeClassifier(max_depth=3, random_state=42)
18
19 # Entrenamos la Inteligencia Artificial con los datos
20 clasificador.fit(X, y)
21
22 # 3. RENDERIZADO VISUAL DEL ARBOL
23 # Configuramos el tamaño del "lienzo"
24 plt.figure(figsize=(10, 6))
25
26 # Dibujamos el grafo estructurado
27 plot_tree(clasificador,
28           feature_names=["Horas Estudio", "Asistencia %"],
29           class_names=["Reprueba", "Aprueba"],
30           filled=True, # Colorea los nodos segun la clase dominante
31           rounded=True, # Bordes redondeados en los nodos
32           impurity=False) # Oculta el indice Gini para mantener la grafica limpia y simple
33
34 # Título y visualizacion de la ventana
35 plt.title("Aplicacion de Teoria de Grafos en IA: Arbol de Decision", fontsize=14, fontweight="bold")
36 plt.tight_layout()
37 plt.show()

```

Figura 19. Ejemplo de árbol de decisión implementado en Python.

Conclusión

La teoría de grafos y árboles demuestra que las matemáticas discretas no son conceptos aislados, sino los planos arquitectónicos del software. Desde la forma en que el teclado predice la siguiente palabra (tries), hasta cómo un servidor recupera un registro entre millones (árboles B) o cómo una inteligencia artificial toma una decisión, los árboles son la estructura que hace posible la computación escalable.

FASE 4: PROYECTO INTEGRADOR - MODELADO Y OPTIMIZACIÓN DE REDES

Análisis de Redes Sociales y Modelado de Estructuras Basado en Grafos

Problemática

El análisis de redes sociales (SNA, por sus siglas en inglés) representa un campo interdisciplinario de importancia estratégica, diseñado para decodificar las estructuras sociales y

las dinámicas sistémicas mediante el rigor de la teoría de grafos. En el entorno actual de macrodatos, el SNA no se limita a la representación visual, sino que constituye un marco analítico para comprender cómo las interacciones individuales consolidan jerarquías y facilitan u obstruyen el flujo de activos dentro de una red. Siguiendo la arquitectura conceptual de Dmitri Goldenberg, el SNA integra la psicología social, la estadística y la matemática para investigar patrones complejos en conjuntos de datos reales. Esta integración permite pasar de la observación anecdótica a la cuantificación precisa de fenómenos estructurales, lo que permite a los científicos de datos modelar comportamientos sistémicos con alta fidelidad.

La distinción entre redes matemáticas aleatorias y redes del mundo real es el pilar de la precisión analítica profesional. Las redes sociales reales no poseen una distribución uniforme; su topología única dicta el comportamiento del sistema. Ignorar estas propiedades estructurales resultaría en modelos predictivos fallidos y decisiones estratégicas erróneas en contextos de negocio o investigación. Este marco conceptual es la base necesaria para desplegar el modelado matemático detallado que define la infraestructura lógica del sistema.

Modelado Matemático

El modelado matemático actúa como la infraestructura lógica que permite transformar abstracciones en métricas cuantificables de relación y jerarquía. En la arquitectura de sistemas complejos, esta lógica es la que permite discernir la influencia real sobre la conectividad aparente.

Componentes Fundamentales del Grafo

Nodos (vértices): representan las entidades del sistema (actores, servidores, países). Poseen propiedades intrínsecas, como peso, tamaño, posición geográfica y etiquetas de clase, y propiedades basadas en la red, como el grado (número de vecinos directos) y la pertenencia a componentes conectados (clústeres).

Aristas (edges): representan la infraestructura de conexión. Poseen propiedades críticas como el peso (intensidad de la relación), la dirección (asimetría del flujo) y la temporalidad.

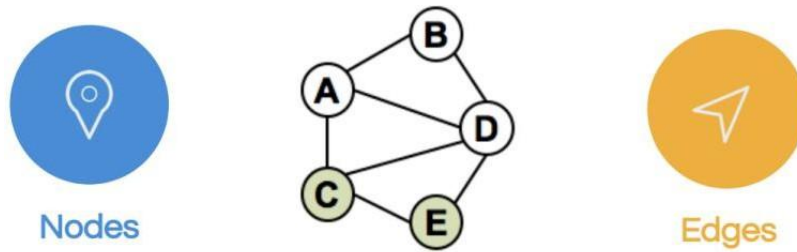


Figura 20. Componentes fundamentales del grafo: nodos y aristas.

Lógica de Centralidad y Aplicaciones Sistémicas

La identificación de “actores clave” se realiza mediante medidas de centralidad que cuantifican diferentes dimensiones de importancia:

Medida de centralidad	Definición y lógica algorítmica	Aplicación sistémica
Grado (degree)	Recuento directo de vecinos o adyacencias de un nodo.	Identificación de conectividad inmediata y popularidad.
Eigenvector / PageRank	Algoritmo iterativo donde la relevancia de un nodo es función de la importancia de sus vecinos.	Ranking de autoridad y relevancia en buscadores web.
Cercanía (closeness)	Medida de proximidad basada en el inverso de la suma de las distancias más cortas a todos los demás nodos.	Optimización de centros de transporte y logística.
Intermediación (betweenness)	Cuantificación de la frecuencia con la que un nodo actúa como puente en los caminos más cortos de la red.	Detección de puntos críticos, cuellos de botella y gatekeepers.

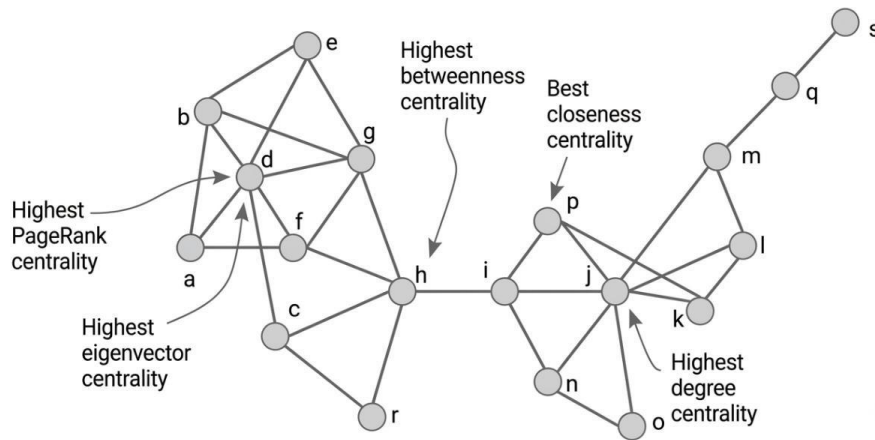


Figura 21. Visualización de medidas de centralidad en una red.

Fenómenos Topológicos del Mundo Real

Las redes reales presentan tres fenómenos característicos que alteran su topología:

- *Mundo pequeño (small world)*: existencia de caminos extremadamente cortos entre cualquier par de nodos, validando la teoría de los seis grados de separación.
- *Libre de escala (scale-free)*: distribución de grado siguiendo una ley de potencia; el sistema se caracteriza por unos pocos “hubs” hiperconectados y una vasta periferia.
- *Homofilia*: tendencia de los nodos a vincularse con pares que comparten atributos similares, lo que deriva en propiedades compartidas entre vecinos y facilita la predicción de comportamientos.

Implementación en Software

La transición del modelo matemático a la ejecución técnica requiere herramientas escalables. Python, mediante la librería NetworkX, permite procesar datos masivos y transformar registros tabulares en arquitecturas de grafos legibles.

Flujo de Trabajo Técnico: El Caso Eurovisión Mediante Python y NetworkX

El proceso de construcción de red para el análisis de votaciones se rige por un pipeline de transformación de datos:

- *Ingesta tabular*: uso de Pandas para la representación inicial de los datos (matriz de adyacencia).
- *Reestructuración de esquema*: aplicación de la función melt para transmutar el formato “ancho” original en un esquema de lista de aristas (edgelist) “largo”. Esta transformación es crítica, ya que la lista de aristas es el estándar de entrada optimizado para motores de grafos.
- *Instanciación del grafo*: construcción de un objeto DiGraph (grafo dirigido) en NetworkX para preservar la asimetría de los votos.

Estrategia de Visualización como Herramienta Diagnóstica

El método estándar `nx.draw_networkx` suele fallar en redes densas al emplear un force-directed layout (diseño basado en fuerzas) que genera diagramas ininteligibles o “hairballs”. Para superar esto, se aplica una estrategia de “dividir y conquistar”:

- *Posicionamiento geográfico*: sustitución de fuerzas abstractas por coordenadas reales.
- *Codificación de atributos*: uso de banderas y colores para identificación inmediata.
- *Jerarquización visual*: el tamaño de nodos y el grosor de aristas se vinculan dinámicamente al peso de los votos.

Resultados

Modelar la difusión de información es fundamental para predecir el alcance de mensajes o la propagación de riesgos, procesos que guardan similitudes matemáticas con el contagio biológico.

Modelos de Difusión Comparados

1. Umbral lineal (linear threshold): la activación ocurre por influencia acumulada. Un nodo cambia de estado solo si la suma ponderada de sus vecinos activos supera un umbral crítico.
2. Cascada independiente (independent cascade): cada nodo activo tiene una probabilidad independiente de activar a sus contactos directos en cada paso de tiempo. Es el estándar para modelar la viralidad pura.

Análisis de Difusión: Caso Game of Thrones

En la validación del modelo sobre la red de personajes de A Storm of Swords, se simuló la propagación de un rumor:

- *Tiempo $t = 0$* : el secreto se encuentra en el conjunto “semilla” inicial, donde Jon Snow es el actor principal de observación.
- *Tiempo $t = 1$ a $t = 2$* : la información se transfiere a los vecinos inmediatos de la semilla (como Tyrion Lannister o Joffrey Baratheon), iniciando la expansión.
- *Tiempo $t = 5$* : el flujo ha penetrado la topología de la red de tal manera que el secreto se ha transformado en conocimiento público global.

Análisis de Optimización y Maximización de Influencia

El problema de maximización de influencia es inherentemente NP-difícil, lo que implica que hallar la solución óptima absoluta es computacionalmente inmanejable en redes de gran escala.

- Las heurísticas basadas en centralidad ofrecen una aproximación eficiente.
- Sin embargo, el análisis de fuerza bruta demostró que la combinación de Robert Baratheon y Khal Drogo como pareja inicial logra una cobertura del 56% de la red, un rendimiento superior que subraya la importancia de la selección estratégica de semillas sobre el simple volumen de conexiones.

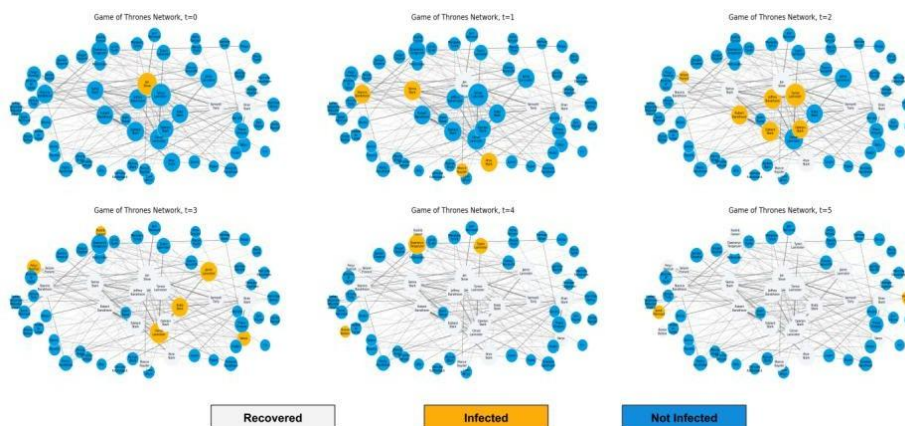


Figura 22. Resultados del análisis de maximización de influencia.

Conclusiones

El análisis de redes sociales se ha consolidado como una herramienta de diagnóstico y predicción indispensable para la gestión de sistemas complejos en la era del big data. Su valor reside en la capacidad de transformar datos relacionales crudos en inteligencia accionable.

Aplicabilidad versátil: las metodologías aquí descritas son directamente transferibles a la maximización de impacto en marketing, la detección de anomalías en redes de fraude y el refinamiento de sistemas de recomendación.

Recomendación estratégica: no existe una métrica de centralidad universal. La arquitectura del análisis debe diseñarse a medida, considerando las propiedades únicas de la red (mundo pequeño, homofilia) y el proceso dinámico que se desea modelar. El rigor en la fase de modelado inicial es lo que garantiza la integridad de las conclusiones finales.

FASE 5: AUTOEVALUACIÓN INDIVIDUAL

A continuación, cada estudiante presenta una autoevaluación individual sobre su proceso de aprendizaje durante las tres unidades de la asignatura, identificando fortalezas, debilidades y metas de mejora para el siguiente ciclo académico.

Karen Lalangui

Unidad 1

Durante el desarrollo de la Unidad 1, el estudio de la lógica proposicional permitió fortalecer el pensamiento crítico y analítico, proporcionando las bases necesarias para abordar futuros temas dentro del área de Matemáticas Discretas. El aprendizaje de los conectores lógicos, las tablas de verdad y los métodos de inferencia facilitó la simplificación de expresiones lógicas, lo que hizo posible diseñar circuitos más eficientes, optimizar el uso de recursos y mejorar el desempeño de los sistemas implementados.

Unidad 2

Durante el desarrollo de la Unidad 2, el estudio del álgebra de Boole permitió comprender cómo los principios de la lógica se aplican directamente al funcionamiento del hardware y al desarrollo de software eficiente. El aprendizaje de las operaciones booleanas, las leyes del álgebra de Boole y las técnicas de simplificación, como los mapas de Karnaugh, fortaleció la capacidad de

analizar y optimizar expresiones lógicas, favoreciendo el diseño de circuitos digitales más rápidos, eficientes y con un menor consumo de recursos. Finalmente, la aplicación de metodologías activas, como el aula invertida, promovió un aprendizaje más participativo, en el que la práctica y la resolución de problemas consolidaron los conocimientos adquiridos y evidenciaron la importancia del álgebra de Boole como base fundamental de la computación moderna.

Unidad 3

Durante el desarrollo de la Unidad 3, el estudio de los grafos y los árboles permitió comprender cómo es posible representar y resolver problemas del mundo real mediante modelos matemáticos. El aprendizaje de estos conceptos fortaleció la capacidad de analizar estructuras de conectividad, jerarquías y relaciones entre elementos, proporcionando las bases para abordar problemas de optimización en áreas como las redes de computadoras, la logística, las telecomunicaciones y la gestión de recursos. Asimismo, el conocimiento de los árboles permitió entender el funcionamiento de estructuras de datos fundamentales, los sistemas de archivos y diversos algoritmos utilizados en inteligencia artificial.

Miguel Rojas

Unidad 1

En la Unidad 1, el análisis detallado de la lógica proposicional y la teoría de conjuntos sentó las bases cognitivas esenciales para estructurar el razonamiento matemático y el pensamiento algorítmico. El dominio de las tablas de verdad, los conectores lógicos y el estudio formal de tautologías, contradicciones y contingencias facilitó la evaluación rigurosa de argumentos y la simplificación de enunciados complejos. Adicionalmente, relacionar estos conceptos con el diseño y optimización de circuitos lógicos evidenció la utilidad práctica de la lógica matemática en la construcción de sistemas digitales más eficientes, permitiendo minimizar el uso de compuertas y mejorar el rendimiento de los diseños lógicos implementados.

Unidad 2

En la Unidad 2, el estudio del álgebra de Boole y los métodos de simplificación de funciones permitió comprender de manera profunda el comportamiento del hardware y la arquitectura de los computadores. A través del análisis de teoremas y leyes booleanas, así como la

resolución práctica de mapas de Karnaugh, se desarrolló la capacidad para diseñar y optimizar sistemas lógicos combinacionales con una reducción significativa en el consumo de recursos de simulación. Esta etapa de aprendizaje práctico y experimental consolidó la habilidad para transformar especificaciones abstractas en soluciones físicas y de software funcionales, demostrando que el álgebra de Boole es el pilar fundamental que sustenta la tecnología y el procesamiento digital de la información.

Unidad 3

En la Unidad 3, el estudio formal de la teoría de grafos y árboles proporcionó las herramientas metodológicas para modelar y resolver problemas complejos de optimización y conectividad en redes reales. Al analizar propiedades clave como los caminos y circuitos de Euler y Hamilton, y al implementar algoritmos fundamentales como el de Dijkstra o Kruskal, se fortaleció la habilidad para evaluar redes de computadoras y sistemas logísticos. Asimismo, la investigación y uso de matrices de adyacencia e incidencia, junto con el análisis de árboles binarios de búsqueda y estructuras de datos jerárquicas, permitieron asimilar de qué manera se gestiona y recupera la información de forma óptima en el desarrollo de software y en sistemas inteligentes complejos.

Andy Ordoñez

Unidad 1

En la Unidad 1, el estudio de la lógica proposicional permitió fortalecer el pensamiento crítico y analítico, proporcionando las bases necesarias para abordar futuros temas dentro del área de Matemáticas Discretas. El aprendizaje de los conectores lógicos y la construcción de tablas de verdad facilitó la simplificación de expresiones lógicas, complementándose con el análisis de proposiciones condicionales y la clasificación de argumentos en tautologías, contradicciones y contingencias. La asimilación de estos conceptos hizo posible comprender cómo se pueden diseñar circuitos más eficientes, optimizar el uso de recursos y mejorar el desempeño de los sistemas implementados.

Unidad 2

En la Unidad 2, el estudio del álgebra de Boole permitió comprender cómo los principios de la lógica se aplican directamente al funcionamiento del hardware y al desarrollo de software eficiente. A través del análisis teórico de sus leyes y la simplificación práctica de expresiones booleanas, se fortaleció la capacidad de analizar y optimizar sistemas lógicos, favoreciendo el diseño de circuitos digitales más rápidos, eficientes y con un menor consumo de recursos. Además, la aplicación de estas bases prácticas promovió un aprendizaje más participativo, en el que la resolución de problemas consolidó los conocimientos adquiridos y evidenció la importancia del álgebra de Boole como pilar de la computación moderna.

Unidad 3

En la Unidad 3, el estudio exhaustivo de grafos y árboles permitió comprender cómo es posible representar y resolver problemas del mundo real mediante modelos matemáticos. Al integrar conceptos como paseos de Euler y Hamilton, algoritmos como el de Dijkstra o Fleury, y el uso de matrices de adyacencia e incidencia, se fortaleció la capacidad de analizar estructuras de conectividad y jerarquías para abordar problemas de optimización en áreas como las redes de computadoras o la logística. Asimismo, el análisis profundo de árboles enraizados y de búsqueda binaria permitió entender el funcionamiento de estructuras de datos fundamentales, sistemas de archivos y algoritmos avanzados aplicados en inteligencia artificial.

David Luna

Unidad 1

La asimilación de la lógica proposicional durante la primera unidad representó un pilar fundamental para la formalización del razonamiento algorítmico y deductivo. La comprensión profunda de las conectivas lógicas, junto con la evaluación sistemática mediante matrices de verdad y la aplicación rigurosa de axiomas de inferencia, propició el desarrollo de destrezas analíticas orientadas a la minimización de funciones proposicionales. Esta competencia abstracta se traduce empíricamente en la capacidad de sintetizar arquitecturas lógicas subyacentes, mitigando la redundancia computacional y optimizando la asignación de recursos para elevar el rendimiento de las infraestructuras tecnológicas.

Unidad 2

La inmersión en el álgebra de Boole a lo largo de la segunda unidad evidenció la transición intrínseca entre la abstracción matemática y la ingeniería de sistemas microelectrónicos. La operatoria booleana, la aplicación sistemática de sus teoremas y el uso de métodos gráficos de simplificación, como los diagramas de Karnaugh, catalizaron la habilidad analítica para refinar topologías lógicas. Esto repercute directamente en la implementación de circuitos combinacionales de alta velocidad y baja disipación de potencia. Adicionalmente, la adopción del paradigma pedagógico del aula invertida fomentó una apropiación empírica del conocimiento, afianzando mediante la heurística y la resolución de problemas el papel del álgebra de Boole como la infraestructura medular del procesamiento de datos contemporáneo.

Unidad 3

El escrutinio de la teoría de grafos y las topologías arbóreas en la tercera unidad demostró la eficacia de la abstracción matemática para el modelado relacional de ecosistemas complejos. La interiorización de estos paradigmas estructurales potenció la capacidad analítica para decodificar matrices de conectividad, esquemas de ruteo y arquitecturas jerárquicas. Este constructo teórico resulta insustituible para la formulación de algoritmos de optimización aplicados en telecomunicaciones, logística de grafos ponderados y topologías de red. De igual modo, la taxonomía de los árboles probó ser crucial para comprender el diseño de estructuras de datos no lineales, los esquemas de indexación en sistemas de archivos y los algoritmos de búsqueda y toma de decisiones utilizados en el campo de la inteligencia artificial.

Preguntas de Control

1. Diferencias entre BFS y DFS

Explique la diferencia entre los algoritmos BFS y DFS. ¿En qué situaciones sería más apropiado utilizar BFS y en cuáles DFS? Argumente con ejemplos de aplicaciones reales en la ciencia de la computación.

Tanto BFS (Breadth-First Search) como DFS (Depth-First Search) son algoritmos para recorrer grafos o árboles, pero la diferencia central está en el orden en que visitan los nodos y en la estructura de datos que usan internamente.

BFS (recorrido en anchura) explora el grafo nivel por nivel: primero visita todos los vecinos directos del nodo de partida, luego los vecinos de esos vecinos, y así sucesivamente. Para lograr esto utiliza una cola (FIFO), de modo que el primer nodo que entra es el primero en procesarse. Esto hace que BFS siempre encuentre el camino más corto (en número de aristas) entre el nodo de origen y cualquier otro nodo, cuando el grafo no tiene pesos.

DFS (recorrido en profundidad) avanza por una rama hasta el final antes de retroceder (backtracking) y explorar otra. Internamente usa una pila (LIFO), ya sea explícita o mediante la pila de llamadas de la recursión. No garantiza el camino más corto, pero es más eficiente en memoria en grafos muy anchos, y es la base natural de muchos algoritmos de backtracking.

¿Cuándo Usar Cada Uno?

BFS conviene cuando se necesita el camino mínimo en un grafo no ponderado, o cuando se quiere explorar “cerca” antes que “lejos”. Ejemplos reales:

- Redes sociales: LinkedIn o Facebook usan BFS para calcular el “grado de separación” entre dos usuarios (amigos de amigos).
- Routers y protocolos de red: para encontrar la ruta con menos saltos entre dos dispositivos.
- Rastreadores web (web crawlers), cuando se quiere indexar primero las páginas más “cercanas” al sitio inicial.
- Algoritmos de GPS en su versión no ponderada, para hallar la ruta con menos intersecciones.

DFS conviene cuando se necesita explorar todas las posibilidades de una rama antes de decidir, o cuando la memoria es limitada. Ejemplos reales:

- Resolución de laberintos y juegos como el Sudoku o las N-reinas (backtracking).
- Detección de ciclos en un grafo (por ejemplo, para evitar deadlocks en sistemas operativos).
- Ordenamiento topológico, usado por compiladores y gestores de paquetes (npm, pip) para resolver el orden correcto de dependencias.
- Recorrido de sistemas de archivos (explorar carpetas y subcarpetas hasta el final antes de pasar a la siguiente).

2. Construcción de un Árbol Binario de Búsqueda (BST)

Construya un árbol binario de búsqueda (BST) insertando los siguientes valores en orden: 45, 25, 65, 15, 35, 55, 75, 10, 20, 30. Muestre el árbol resultante y realice los recorridos inorden, preorden y posorden. ¿Qué utilidad tiene cada recorrido en la programación?

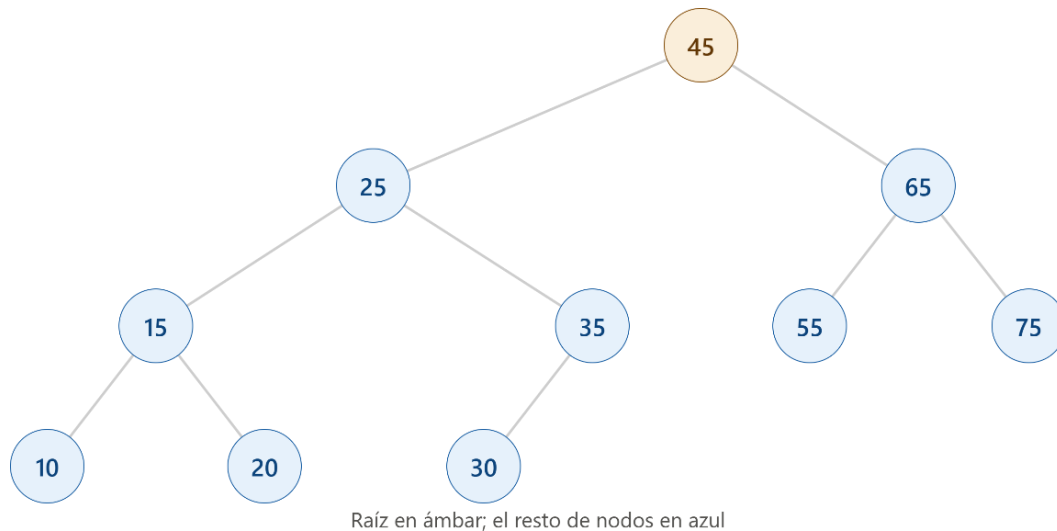


Figura 23. Árbol binario de búsqueda resultante.

Recorridos

Preorden (raíz → izquierda → derecha): 45, 25, 15, 10, 20, 35, 30, 65, 55, 75.

Inorden (izquierda → raíz → derecha): 10, 15, 20, 25, 30, 35, 45, 55, 65, 75.

Postorden (izquierda → derecha → raíz): 10, 20, 15, 30, 35, 25, 55, 75, 65, 45.

¿Para Qué Sirve Cada Recorrido en Programación?

Inorden: en un BST siempre devuelve los elementos ordenados de menor a mayor (como se ve arriba). Se usa para verificar que un árbol es válido como BST, para generar listas ordenadas sin tener que llamar a un algoritmo de ordenamiento aparte, y para consultas por rango.

Preorden: se usa para clonar o serializar un árbol (guardarlo en un archivo o enviarlo por red), porque al reconstruirlo en el mismo orden se recupera la misma estructura. También sirve para evaluar expresiones en notación prefija (notación polaca).

Postorden: se usa para eliminar o liberar memoria de un árbol de forma segura, porque procesa los hijos antes que el padre (así no se pierde la referencia a los hijos antes de borrarlos).

También se usa para evaluar expresiones en notación postfija y para calcular tamaños de carpetas de abajo hacia arriba en un sistema de archivos.

3. Teoría de Grafos y Brecha Digital en Ecuador

Analice cómo la teoría de grafos puede contribuir a reducir la brecha digital en Ecuador. Proponga un modelo de red de conectividad para una comunidad rural de la provincia de Loja, utilizando los conceptos y algoritmos aprendidos en esta unidad.

La teoría de grafos puede modelar la infraestructura de conectividad como un grafo ponderado, donde:

- **Nodos** = comunidades, escuelas, centros de salud, antenas o repetidores.
- **Aristas** = posibles enlaces de conexión (fibra, radioenlace, wifi de largo alcance, satélite), con un peso que representa el costo de instalación (distancia, terreno montañoso, energía disponible).

Propuesta de Modelo para una Comunidad Rural de Loja

3. **Modelado del grafo:** se representa cada caserío, escuela y centro de salud de la parroquia como un nodo, y cada posible enlace inalámbrico (línea de vista entre cerros) como una arista ponderada según la distancia y el costo de instalación de la antena.
4. **Árbol de expansión mínima (MST) con Kruskal o Prim:** se aplica un algoritmo de árbol de expansión mínima para encontrar la combinación de enlaces que conecta a todas las comunidades con el menor costo total de infraestructura. Esto evita construir más torres de las necesarias, algo clave en zonas rurales con presupuesto limitado.
5. **Nodo central (hub):** se ubica el nodo principal en la cabecera parroquial, que sería el único punto conectado a la red troncal de fibra óptica o a un enlace satelital, y desde ahí se distribuye la señal en cascada hacia las comunidades más alejadas siguiendo el árbol calculado.
6. **Análisis de puntos críticos (bridges y centralidad de intermediación):** se identifican qué enlaces son “puentes” (si se caen, dejan incomunicada a una comunidad completa) usando conceptos de conectividad de grafos, y en esos puntos

se propone un enlace redundante (un segundo camino) para que la red no dependa de una sola antena.

7. **Verificación con BFS:** una vez armada la red, se usa BFS desde el nodo central para confirmar que todas las comunidades son alcanzables y para medir cuántos “saltos” (repetidores) le toma a la señal llegar a cada escuela, lo cual sirve para estimar la latencia.

Impacto esperado: este modelo permitiría llevar internet a escuelas y centros de salud rurales de Loja con la menor inversión posible, priorizando primero los nodos con mayor centralidad (los que conectan a más comunidades), lo que reduciría la brecha digital al habilitar acceso a educación virtual, telemedicina y trámites en línea en zonas que hoy dependen únicamente de la señal celular, muchas veces intermitente por la geografía montañosa de la provincia.

Conclusiones

Karen Lalangui

El desarrollo de esta unidad permitió consolidar los conocimientos fundamentales sobre la teoría de grafos y la teoría de árboles, comprendiendo su importancia como base para el diseño y análisis de algoritmos en las ciencias de la computación. A través del estudio de los diferentes tipos de grafos, sus representaciones y los algoritmos de búsqueda BFS, DFS y Dijkstra, se fortaleció la capacidad para modelar y resolver problemas relacionados con redes y caminos óptimos. Asimismo, el análisis de los árboles, sus propiedades, recorridos y aplicaciones permitió reconocer su papel en la organización eficiente de la información y en el desarrollo de estructuras de datos. Además, el uso de herramientas como Graph Online y Gephi facilitó la representación y visualización de grafos, favoreciendo una mejor comprensión de los conceptos teóricos mediante la práctica.

Miguel Rojas

El estudio y desarrollo de la Unidad 3 demostró de manera contundente que la teoría de grafos y árboles no es solo una abstracción matemática, sino la infraestructura conceptual sobre la cual se cimienta la ingeniería de software moderna. El análisis práctico de algoritmos de optimización de rutas y redes, complementado con la representación matricial de estructuras de

datos, permitió consolidar la capacidad de traducir problemas complejos de conectividad física o lógica en modelos computacionales altamente eficientes. En definitiva, esta unidad cerró el ciclo de aprendizaje evidenciando que el éxito en el diseño de algoritmos avanzados y la gestión jerárquica de la información depende directamente de un dominio sólido de las estructuras matemáticas discretas.

Andy Ordoñez

El desarrollo de la Unidad 3 permitió consolidar un entendimiento profundo sobre la teoría de grafos y árboles como pilares matemáticos esenciales para las ciencias de la computación. El estudio detallado de los grafos, desde su caracterización, paseos y circuitos de Euler y Hamilton, hasta su representación práctica mediante matrices de adyacencia e incidencia y el análisis de algoritmos clave como Fleury y Dijkstra, fortaleció la capacidad para modelar redes complejas y resolver problemas de optimización de rutas de manera lógica y estructurada. Asimismo, la exploración de la teoría de árboles, incluyendo el análisis de árboles enraizados, los códigos de prefijos, los árboles de búsqueda binaria y sus respectivos métodos de recorrido, evidenció la enorme utilidad de estas estructuras no lineales en la organización jerárquica y el acceso eficiente a la información. En definitiva, esta unidad demostró de manera contundente que el dominio práctico de estas estructuras discretas y sus algoritmos asociados es fundamental para el diseño de soluciones de software robustas, eficientes y escalables.

David Luna

La progresión académica de esta unidad posibilitó la asimilación rigurosa de los postulados inherentes a la teoría de grafos y las topologías arbóreas, cimentando su relevancia ineludible en el modelado, diseño y evaluación de complejidad algorítmica dentro de las ciencias computacionales. Mediante el escrutinio taxonómico de los grafos, sus representaciones matriciales y la ejecución empírica de heurísticas de exploración, tales como las búsquedas no informadas (BFS y DFS) y el algoritmo de rutas mínimas de Dijkstra, se potenció la competencia analítica para abstraer arquitecturas de red y computar trayectorias de costo óptimo.

Paralelamente, la disección técnica de las estructuras arbóreas, abarcando sus propiedades intrínsecas, algoritmos de recorrido y casos de uso, evidenció su función crítica en la indexación jerárquica y en la arquitectura de estructuras de almacenamiento altamente eficientes. De manera

complementaria, la integración instrumental de software especializado, como Graph Online y Gephi, propició la renderización y el análisis topológico de redes complejas, traduciendo constructos matemáticos abstractos en representaciones visuales que catalizaron la asimilación empírica del marco teórico.

Referencias

- Levin, O. (s. f.). *Discrete mathematics: An open introduction* (Cap. 4, Graph Theory). LibreTexts.
[https://math.libretexts.org/Bookshelves/Combinatorics_and_Discrete_Mathematics/Discrete_Mathematics_\(Levin\)/4%3A_Graph_Theory/4.E%3A_Graph_Theory_\(Exercises\)](https://math.libretexts.org/Bookshelves/Combinatorics_and_Discrete_Mathematics/Discrete_Mathematics_(Levin)/4%3A_Graph_Theory/4.E%3A_Graph_Theory_(Exercises))
- Levin, O. (s. f.). *4.4: Euler paths and circuits*. En *Discrete mathematics: An open introduction*. LibreTexts.
[https://math.libretexts.org/Bookshelves/Combinatorics_and_Discrete_Mathematics/Discrete_Mathematics_\(Levin\)/4%3A_Graph_Theory/4.4%3A_Euler_Paths_and_Circuits](https://math.libretexts.org/Bookshelves/Combinatorics_and_Discrete_Mathematics/Discrete_Mathematics_(Levin)/4%3A_Graph_Theory/4.4%3A_Euler_Paths_and_Circuits)
- Universidad Nacional de Loja. (2026). *Teoría de grafos [Video]*. YouTube.
<https://www.youtube.com/watch?v=gBueumPIAg4>
- Villalpando Becerra, J. F. (s. f.). *Matemáticas discretas: Aplicaciones y ejercicios*. FlipHTML5.
https://fliphtml5.com/hchjv/yovx/Matemáticas_discretas__José_Francisco_Villalpando_Becerra-LIBROSVIRTUAL/